Common errors A limitation of subroutines on E20 is that the value of `$7` must be preserved for the duration of the subroutine. If the subroutine modifies `$7`, then the `jr` instruction may return to an unintended location.

A corollary of the above limitation is that a subroutine cannot call another subroutine. Since the `jal` instruction modifies `$7`, this makes recursion awkward on E20.

Both of these limitation have workarounds. For example, a subroutine could save the value of `$7` into another register or into memory. It would then be free to manipulate `$7`, including by invoking other subroutines, as long as it restores the original value of `$7` before it returns.

2.6 E15 vs E20

The E20 processor can be considered a more powerful version of the E15. Although they have many features in common and their instruction sets are similar, the E20 expands some capabilities.

	E15	E20
registers	four 4-bit registers	seven 16-bit registers
memory	16 12-bit cells of read-only instruction memory	8192 16-bit cells of instruction/-data memory
instruction format	all instructions have 4-bit opcode field, two 2-bit register fields, and 4-bit immediate field	three formats: • 3-bit opcode field, three 3-bit register fields, 4-bit immediate field • 3-bit opcode field, two 3-bit register fields, 7-bit immediate field • 3-bit opcode field, 13-bit immediate field

3 Instruction set

Here we discuss the instructions that form the E20 instruction set. We categorize the instructions by their format, which is based on the number of register arguments. The format determines how many bits are allocated to each argument.

In addition to instructions proper, we discuss *pseudo-instructions*, which are assembly mnemonics that are translated to another instruction. Finally, we cover *assembler directives*, which change the behavior of the assembler but do not correspond to any specific instruction.

In the following subsections, we give the following information about each instruction:

- the instruction name and syntax — The header of each subsection gives the assembly language syntax.
- the instruction format — Within each 16-bit instruction, we show which bits must have which values.
- the informal behavior — We give a brief prose description of the behavior of the instruction when executed.
- the formal behavior — We use a symbolic notation to express the behavior of the instruction. Here, we use the following symbols:
  - `<-`: indicates that the location on the left will be updated with the value on the right
  - `$reg` (and variants, such as `$regA`, `$regSrc`, etc): the instruction's register argument

- `imm`: the instruction's immediate argument
- `R`: the processor's register file, indexed by the name of a register, $0 \ldots 7$
- `Mem`: the processor's memory, indexed by a memory address
- `pc`: the program counter

Unless otherwise specified, immediate values can be positive, negative, or zero and are represented in 2's complement. Unless otherwise specified, all instructions increment the program counter.

3.1 Instructions with three register arguments

3.1.1 add $regDst, $regSrcA, $regSrcB

15 14 13	12 11 10	9 8 7	6 5 4	3 2 1 0
3 bits	3 bits	3 bits	3 bits	4 bits
000	regSrcA	regSrcB	regDst	0000

Example: `add $1, $0, $5`
Adds the value of registers `$regSrcA` and `$regSrcB`, storing the sum in `$regDst`.
Symbolically: `R[regDst] <- R[regSrcA] + R[regSrcB]`

3.1.2 sub $regDst, $regSrcA, $regSrcB

15 14 13	12 11 10	9 8 7	6 5 4	3 2 1 0
3 bits	3 bits	3 bits	3 bits	4 bits
000	regSrcA	regSrcB	regDst	0001

Mnemonic: *Subtract*
Example: `sub $1, $0, $5`
Subtracts the value of register `$regSrcB` from `$regSrcA`, storing the difference in `$regDst`.
Symbolically: `R[regDst] <- R[regSrcA] - R[regSrcB]`

3.1.3 or $regDst, $regSrcA, $regSrcB

15 14 13	12 11 10	9 8 7	6 5 4	3 2 1 0
3 bits	3 bits	3 bits	3 bits	4 bits
000	regSrcA	regSrcB	regDst	0010

Example: `or $1, $0, $5`
Calculates the bitwise OR of the value of registers `$regSrcA` and `$regSrcB`, storing the result in `$regDst`.
Symbolically: `R[regDst] <- R[regSrcA] | R[regSrcB]`

3.1.4 and $regDst, $regSrcA, $regSrcB

15 14 13	12 11 10	9 8 7	6 5 4	3 2 1 0
3 bits	3 bits	3 bits	3 bits	4 bits
000	regSrcA	regSrcB	regDst	0011

Example: `and $1, $2, $5`
Calculates the bitwise AND of the value of registers `$regSrcA` and `$regSrcB`, storing the result in `$regDst`.
Symbolically: `R[regDst] <- R[regSrcA] & R[regSrcB]`

3.1.5 slt $regDst, $regSrcA, $regSrcB

15 14 13	12 11 10	9 8 7	6 5 4	3 2 1 0
3 bits	3 bits	3 bits	3 bits	4 bits
000	regSrcA	regSrcB	regDst	0100

Mnemonic: *Set if less than*
Example: `slt $1, $2, $5`
Compares the value of `$regSrcA` with `$regSrcB`, setting `$regDst` to 1 if `$regSrcA` is less than `$regSrcB`, and to 0 otherwise.

The comparison performed is unsigned, meaning that the two operands are treated as unsigned 16-bit integers, not 2's complement integers. Therefore, `0x0000 < 0xFFFF`.
Symbolically: `R[regDst] <- (R[regSrcA] < R[regSrcB]) ? 1 : 0`

3.1.6 jr $reg

15 14 13	12 11 10	9 8 7	6 5 4	3 2 1 0
3 bits	3 bits	3 bits	3 bits	4 bits
000	reg	000	000	1000

Mnemonic: *Jump to register*
Example: `jr $1`
Jumps unconditionally to the memory address in `$reg`.

The jump destination is expressed as an absolute address. All 16 bits of the value of `$reg` are stored into the program counter.
Symbolically: `pc <- R[reg]`

3.2 Instructions with two register arguments

3.2.1 slti $regDst, $regSrc, imm

15 14 13	12 11 10	9 8 7	6 5 4 3 2 1 0
3 bits	3 bits	3 bits	7 bits
111	regSrc	regDst	imm

Mnemonic: *Set if less than, immediate*
Example: `slti $1, $2, some_label`
Example: `slti $1, $2, 30`
Compares the value of `$regSrc` with sign-extended `imm`, setting `$regDst` to 1 if `$regSrc` is less than `imm`, and to 0 otherwise.

The comparison performed is unsigned, meaning that the two operands are treated as unsigned 16-bit integers, not 2's complement integers. Therefore, `0x0000 < 0xFFFF`. This is true even though the argument is expressed as a 7-bit signed number.
Symbolically: `R[regDst] <- (R[regSrc] < imm) ? 1 : 0`

3.2.2 lw $regDst, imm($regAddr)

15 14 13	12 11 10	9 8 7	6 5 4 3 2 1 0
3 bits	3 bits	3 bits	7 bits
100	regAddr	regDst	imm

Mnemonic: *Load word*
Example: `lw $1, some_label($2)`

Example: `lw $1, 5($2)`
Example: `lw $1, -1($2)`

Calculates a memory pointer by summing the signed number `imm` and the value `$regAddr`, and loads the value from that address, storing it in `$regDst`.

The memory address is interpreted as an absolute address. The least significant 13 bits of the value of `$regAddr + imm` are used to index into memory.

Symbolically: `R[regDst] <- Mem[R[regAddr] + imm]`

3.2.3 sw $regSrc, imm($regAddr)

15 14 13	12 11 10	9 8 7	6 5 4 3 2 1 0
3 bits	3 bits	3 bits	7 bits
101	regAddr	regSrc	imm

Mnemonic: *Store word*

Example: `sw $1, some_label($2)`
Example: `sw $1, 5($2)`
Example: `sw $1, -1($2)`

Calculates a memory pointer by summing the signed number `imm` and the value `$regAddr`, and stores the value in `$regSrc` to that memory address.

The memory address is interpreted as an absolute address. The least significant 13 bits of the value of `$regAddr + imm` are used to index into memory.

Symbolically: `Mem[R[regAddr] + imm] <- R[regSrc]`

3.2.4 jeq $regA, $regB, imm

15 14 13	12 11 10	9 8 7	6 5 4 3 2 1 0
3 bits	3 bits	3 bits	7 bits
110	regA	regB	rel_imm

where rel_imm = imm - pc - 1

Mnemonic: *Jump if equal*

Example: `jeq $1, $0, some_label`
Example: `jeq $2, $3, 23` (jumps to address 23)

Compares the value of `$regA` with `$regB`. If the values are equal, jumps to the memory address identified by the address `imm`, which is encoded as the signed number `rel_imm`.

The jump destination, `imm`, is encoded as a relative address, `rel_imm`. That is, when a jump is performed, the value `rel_imm` is sign-extended and added to the successor value of the program counter. Therefore, the actual address that will be jumped to is equal to the current program counter plus one plus the immediate value. This means that when encoding a `jeq` instruction to machine code, the field in the least significant seven bits must be the difference between the desired destination and one plus the program counter.

Symbolically: `pc <- (R[regA] == R[regB]) ? pc+1+rel_imm : pc+1`

3.2.5 addi $regDst, $regSrc, imm

15 14 13	12 11 10	9 8 7	6 5 4 3 2 1 0
3 bits	3 bits	3 bits	7 bits
001	regSrc	regDst	imm

Mnemonic: *Add immediate*

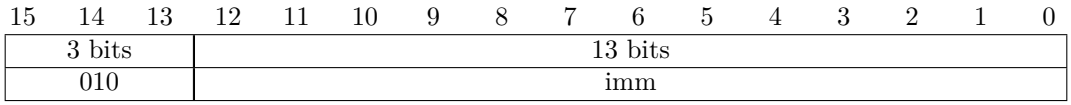
Example: `addi $1, $0, some_label`
Example: `addi $2, $2, -5`

Adds the value of register `$regSrc` and the signed number `imm`, storing the sum in `$regDst`.

Symbolically: `R[regDst] <- R[regSrc] + imm`

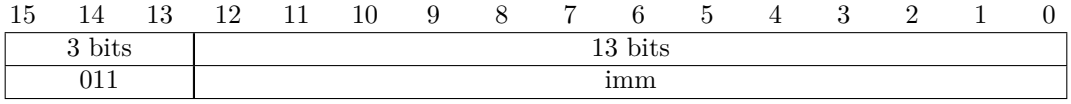
3.3 Instructions with no register arguments

3.3.1 j imm



Mnemonic: *Jump*
Example: `j some_label`
Example: `j 42`
Jumps unconditionally to the memory address `imm`.
The jump destination is expressed as a non-negative absolute address. All 13 bits of `imm` are stored into the program counter, while the most significant 3 bits of the program counter will be set to zero.
Symbolically: `pc <- imm`

3.3.2 jal imm



Mnemonic: *Jump and link*
Example: `jal some_label`
Example: `jal 42`
Stores the memory address of the next instruction in sequence in register `$7`, then jumps unconditionally to the memory address `imm`.
The jump destination is expressed as a non-negative absolute address. All 13 bits of `imm` are stored into the program counter, while the most significant 3 bits of the program counter will be set to zero.
Symbolically: `R[7] <- pc+1; pc <- imm`

3.4 Pseudo-instructions

Pseudo-instructions, unlike proper instructions, do not have their own unique encoding in machine language. Instead, pseudo-instructions are a shorthand form of expressing more complicated assembly instructions. The assembler will translate a pseudo-instruction into an actual instruction, which will then be assembled normally.

3.4.1 movi $reg, imm

Mnemonic: *Move immediate*
Example: `movi $2, 55`
Example: `movi $7, some_label`
Copies the value `imm` to the register `$reg`.
The `movi $reg, imm` instruction is translated by the assembler as `addi $reg, $0, imm`.

3.4.2 nop

Mnemonic: *No operation*
Performs no operation, other than incrementing the program counter.
The `nop` instruction is translated by the assembler as `add $0, $0, $0`.

3.4.3 halt

Performs no operation at all. The program counter is not incremented, resulting in an infinite loop. By convention, represents the end of the program.

The `halt` instruction is translated by the assembler as an unconditional jump (`j`) to the current memory location.

3.5 Assembler directives

Directives are not processor instructions, but rather commands to the assembler itself.

3.5.1 .fill imm

Example: `.fill some_label`

Example: `.fill 42`

Inserts a 16-bit immediate value directly into memory at the current location. This directive instructs the assembler to put a number into the place where an instruction would normally be. The immediate value may be specified numerically (positive, negative, or zero) or with a label.

3.6 Undefined bit patterns

Any bit pattern not covered by any of the previous sections is not valid machine code and its interpretation is undefined.

4 Examples

Here are some examples of E20 assembly language programs. Further examples, including the machine code translation, are available on the class website.

4.1 Math

```
# add, addi, and sub work as you expect, except
# they take three arguments. The first argument
# is the destination, the other two are the
# sources.
# There is no subi. Instead you have to use addi
# with a negative immediate.
addi $1, $0, 5      # $1 := 5
addi $2, $1, -2     # $2 := $1 + (-2)
add $3, $1, $2      # $3 := $1 + $2

# Notice that $0 is special, because it is always
# zero. So using $0 as a source lets us effectively
# do a movi, as in the first instruction below.
addi $4, $0, 55     # $4 := 55
sub $5, $4, $1      # $5 := $4 - $1

# We also have bitwise operators AND and OR.
# (but not ori and andi). As above, the first
# operand is the destination register.
or $6, $2, $5
```